

UNIT-1 LAB EXERCISE

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int palindrome(char str[])
5 {
6     int i = 0;
7     int j = strlen(str) - 1;
8
9     while (i < j)
10     {
11         if (str[i] != str[j])
12             return 0;
13         i++;
14         j--;
15     }
16     return 1;
17 }
18
19 int main()
20 {
21     int n, i;
22     printf("Enter number of words: ");
23     scanf("%d", &n);
24
25     char words[n][100];
26
27     printf("Enter the words:\n");
28     for (i = 0; i < n; i++)
29         scanf("%s", words[i]);
30
31     for (i = 0; i < n; i++)
32     {
33         if (palindrome(words[i]))
34         {
35             printf("First Palindrome: %s", words[i]);
36             return 0;
37         }
38     }
39
40     printf("No palindrome found.");
41     return 0;
42 }
```

5
abc
car
ada

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.808 Mb

Sample Input

5
abc
car
ada
racecar
cool

Your Output

Enter number of words: Enter the words:
First Palindrome: ada

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, m, i, j;
6     int answer1 = 0, answer2 = 0;
7
8     printf("Enter size of first array: ");
9     scanf("%d", &n);
10
11     int nums1[n];
12
13     printf("Enter elements:\n");
14     for(i = 0; i < n; i++)
15         scanf("%d", &nums1[i]);
16
17     printf("Enter size of second array: ");
18     scanf("%d", &m);
19
20     int nums2[m];
21
22     printf("Enter elements:\n");
23     for(i = 0; i < m; i++)
24         scanf("%d", &nums2[i]);
25
26     for(i = 0; i < n; i++)
27     {
28         for(j = 0; j < m; j++)
29         {
30             if(nums1[i] == nums2[j])
31             {
32                 answer1++;
33                 break;
34             }
35         }
36     }
37
38     for(i = 0; i < m; i++)
```

4
3
2
1
2

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.712 Mb

Sample Input

2
3
2
1
2

Your Output

Enter size of first array: Enter elements:
Enter size of second array: Enter elements:
[1, 1]

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j \leq \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j, k;
6
7     printf("Enter size of array: ");
8     scanf("%d", &n);
9
10    int arr[n];
11
12    printf("Enter elements:\n");
13    for(i = 0; i < n; i++)
14        scanf("%d", &arr[i]);
15
16    int sum = 0;
17    for(i = 0; i < n; i++)
18    {
19        int freq[1001] = {0};
20        int distinct = 0;
21
22        for(j = i; j < n; j++)
23        {
24            if(freq[arr[j]] == 0)
25                distinct++;
26            freq[arr[j]]++;
27
28            sum += distinct * distinct;
29        }
30    }
31
32    printf("Sum = %d", sum);
33
34    return 0;
35 }
```

Run

3
1 2 1

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.804 Mb

Sample Input

3
1 2 1

Your Output

Enter size of array: Enter elements:
Sum = 15

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs `(i, j)` where $0 \leq i \leq j \leq n$, such that `nums[i] == nums[j]` and `(i * j)` is divisible by `k`.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, k;
6
7     printf("Enter size: ");
8     scanf("%d", &n);
9
10    int arr[n];
11
12    printf("Enter elements:\n");
13    for(int i = 0; i < n; i++)
14        scanf("%d", &arr[i]);
15
16    printf("Enter k: ");
17    scanf("%d", &k);
18
19    int count = 0;
20
21    for(int i = 0; i < n; i++)
22    {
23        for(int j = i + 1; j < n; j++)
24        {
25            if(arr[i] == arr[j] && ((i * j) % k == 0))
26                count++;
27        }
28    }
29
30    printf("Pairs = %d", count);
31
32    return 0;
33 }
```

Run

7
3 1 2 2 2 1 3
2

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.744 Mb

Sample Input

7
3 1 2 2 2 1 3
2

Your Output

Enter size: Enter elements:
Enter k: Pairs = 4

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

- 1) Input: {1, 2, 3, 4, 5} Expected Output: 5
- 2) Input: {7, 7, 7, 7, 7} Expected Output: 7
- 3) Input: {-10, 2, 3, -4, 5} Expected Output: 5

The screenshot shows a C program in a code editor with a dark theme. The program is designed to find the maximum element in an array. It includes `<stdio.h>` and defines a `main` function. Inside `main`, it prompts the user to enter the size of the array (`n`) and then the elements of the array. It checks if `n` is zero and prints "Array is empty" if so. Otherwise, it declares an array `arr` of size `n`, reads the elements, and then iterates through the array to find the maximum value. The output of the program is shown on the right, indicating it was successfully executed with a time of 0.0000 secs and memory of 1.728 Mb. The sample input and output are also displayed.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n;
6     printf("Enter size: ");
7     scanf("%d", &n);
8
9     if(n == 0)
10    {
11        printf("Array is empty");
12        return 0;
13    }
14
15    int arr[n];
16
17    printf("Enter elements:\n");
18    for(int i = 0; i < n; i++)
19        scanf("%d", &arr[i]);
20
21    int max = arr[0];
22    for(int i = 1; i < n; i++)
23    {
24        if(arr[i] > max)
25            max = arr[i];
26    }
27
28    printf("Maximum = %d", max);
29
30    return 0;
31 }
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.728 Mb

Sample Input

5
1 2 3 4 5

Your Output

Enter size: Enter elements:
Maximum = 5

6. You have an algorithm that process a list of numbers. It firstly sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

The screenshot shows a C program in a code editor with a dark theme. The program is designed to sort an array using bubble sort and then find the maximum element. It includes `<stdio.h>` and defines a `main` function. Inside `main`, it prompts the user to enter the size of the array (`n`) and then the elements of the array. It checks if `n` is zero and prints "Array is empty" if so. Otherwise, it declares an array `arr` of size `n`, reads the elements, and then sorts the array using bubble sort. After sorting, it finds the maximum element in the sorted array. The output of the program is shown on the right, indicating it was successfully executed with a time of 0.0000 secs and memory of 1.796 Mb. The sample input and output are also displayed.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j, temp;
6
7     printf("Enter size of array: ");
8     scanf("%d", &n);
9
10    if (n == 0)
11    {
12        printf("Array is empty");
13        return 0;
14    }
15
16    int arr[n];
17
18    printf("Enter elements:\n");
19    for (i = 0; i < n; i++)
20        scanf("%d", &arr[i]);
21
22    // Bubble Sort
23    for (i = 0; i < n - 1; i++)
24    {
25        for (j = 0; j < n - i - 1; j++)
26        {
27            if (arr[j] > arr[j + 1])
28            {
29                temp = arr[j];
30                arr[j] = arr[j + 1];
31                arr[j + 1] = temp;
32            }
33        }
34    }
35
36    printf("Sorted Array: ");
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.796 Mb

Sample Input

5
4 2 7 1 5

Your Output

Enter size of array: Enter elements:
Sorted Array: 1 2 4 5 7
Maximum Element = 7

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

The screenshot shows a C program in a code editor. The program prompts the user to enter the size of an array and then the elements. It then prints out the unique elements. The output window shows the input size 8 and elements 3 7 3 5 2 5 9 2, and the unique elements 3 7 5 2 9. The status is 'Successfully executed' with a time of 0.0000 secs and memory of 1.88 Mb.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j, found;
6     printf("Enter size of array: ");
7     scanf("%d", &n);
8     int arr[n];
9     printf("Enter elements:\n");
10    for (i = 0; i < n; i++)
11        scanf("%d", &arr[i]);
12    printf("Unique Elements: ");
13    for (i = 0; i < n; i++)
14    {
15        found = 0;
16        for (j = 0; j < i; j++)
17        {
18            if (arr[i] == arr[j])
19            {
20                found = 1;
21                break;
22            }
23        }
24        if (!found)
25            printf("%d ", arr[i]);
26    }
27    return 0;
28 }
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.88 Mb

Sample Input

8
3 7 3 5 2 5 9 2

Your Output

Enter size of array: Enter elements:
Unique Elements: 3 7 5 2 9

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

The screenshot shows a C program in a code editor. The program prompts the user to enter the size of an array and then the elements. It then sorts the array using bubble sort and prints the sorted array. The output window shows the input size 5 and elements 64 34 25 12 22, and the sorted array 12 22 25 34 64. The status is 'Successfully executed' with a time of 0.0000 secs and memory of 1.864 Mb.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j, temp;
6     printf("Enter size of array: ");
7     scanf("%d", &n);
8     int arr[n];
9     printf("Enter elements:\n");
10    for (i = 0; i < n; i++)
11        scanf("%d", &arr[i]);
12    for (i = 0; i < n - 1; i++)
13    {
14        for (j = 0; j < n - i - 1; j++)
15        {
16            if (arr[j] > arr[j + 1])
17            {
18                temp = arr[j];
19                arr[j] = arr[j + 1];
20                arr[j + 1] = temp;
21            }
22        }
23    }
24    printf("Sorted Array: ");
25    for (i = 0; i < n; i++)
26        printf("%d ", arr[i]);
27    return 0;
28 }
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.864 Mb

Sample Input

5
64 34 25 12 22

Your Output

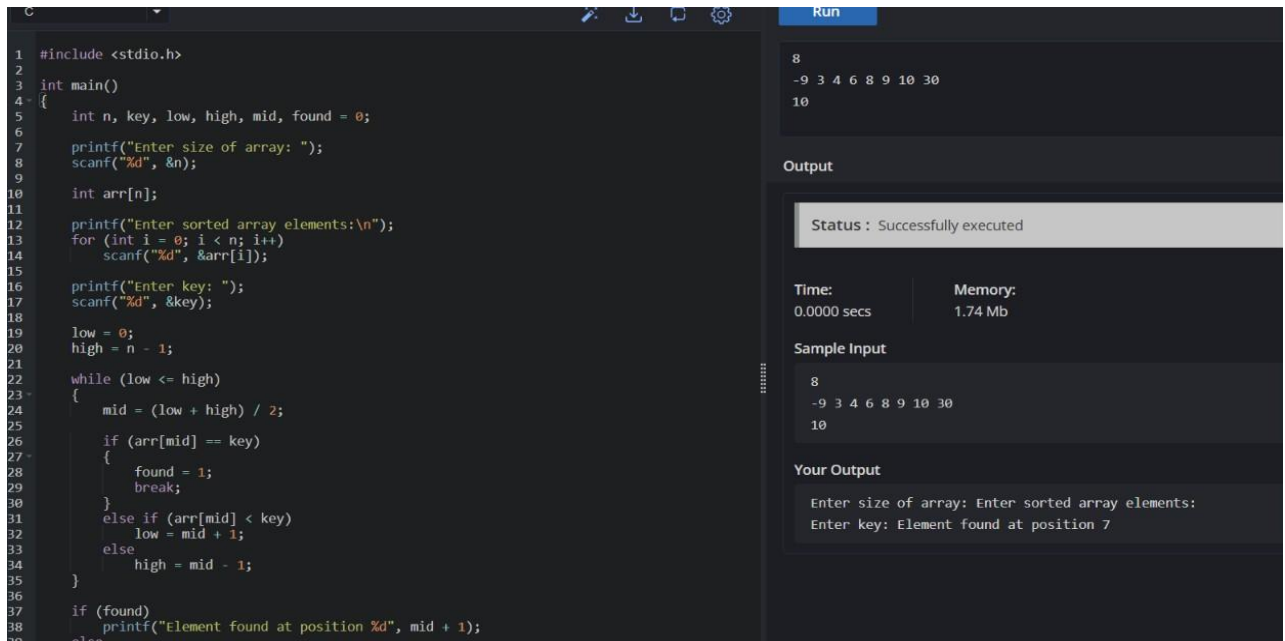
Enter size of array: Enter elements:
Sorted Array: 12 22 25 34 64

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example $X = \{3, 4, 6, -9, 10, 8, 9, 30\}$ KEY=10

Output: Element 10 is found at position 5



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, key, low, high, mid, found = 0;
6
7     printf("Enter size of array: ");
8     scanf("%d", &n);
9
10    int arr[n];
11
12    printf("Enter sorted array elements:\n");
13    for (int i = 0; i < n; i++)
14        scanf("%d", &arr[i]);
15
16    printf("Enter key: ");
17    scanf("%d", &key);
18
19    low = 0;
20    high = n - 1;
21
22    while (low <= high)
23    {
24        mid = (low + high) / 2;
25
26        if (arr[mid] == key)
27        {
28            found = 1;
29            break;
30        }
31        else if (arr[mid] < key)
32            low = mid + 1;
33        else
34            high = mid - 1;
35    }
36
37    if (found)
38        printf("Element found at position %d", mid + 1);
39    else
```

8
-9 3 4 6 8 9 10 30
10

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.74 Mb

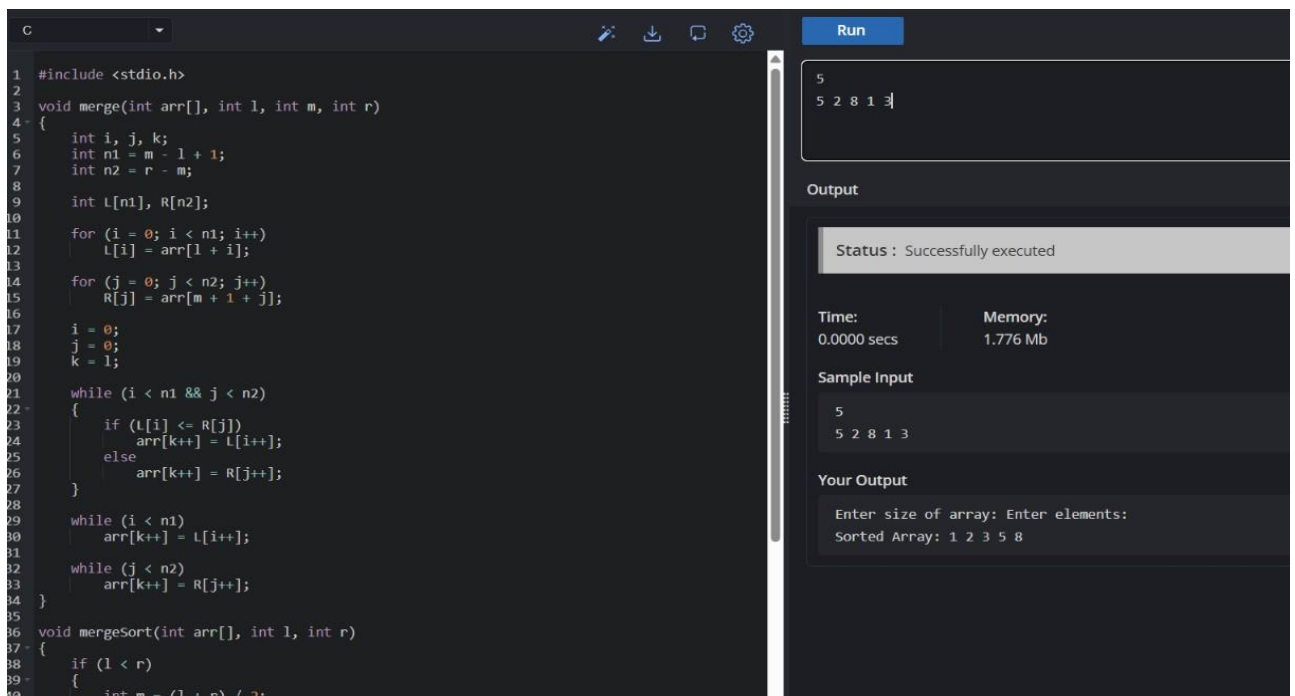
Sample Input

8
-9 3 4 6 8 9 10 30
10

Your Output

Enter size of array: Enter sorted array elements:
Enter key: Element found at position 7

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.



```
1 #include <stdio.h>
2
3 void merge(int arr[], int l, int m, int r)
4 {
5     int i, j, k;
6     int n1 = m - l + 1;
7     int n2 = r - m;
8
9     int L[n1], R[n2];
10
11    for (i = 0; i < n1; i++)
12        L[i] = arr[l + i];
13
14    for (j = 0; j < n2; j++)
15        R[j] = arr[m + 1 + j];
16
17    i = 0;
18    j = 0;
19    k = l;
20
21    while (i < n1 && j < n2)
22    {
23        if (L[i] <= R[j])
24            arr[k++] = L[i++];
25        else
26            arr[k++] = R[j++];
27    }
28
29    while (i < n1)
30        arr[k++] = L[i++];
31
32    while (j < n2)
33        arr[k++] = R[j++];
34 }
35
36 void mergeSort(int arr[], int l, int r)
37 {
38     if (l < r)
39     {
40         int m = (l + r) / 2;
```

5
5 2 8 1 3

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.776 Mb

Sample Input

5
5 2 8 1 3

Your Output

Enter size of array: Enter elements:
Sorted Array: 1 2 3 5 8

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

- Input: $m=2, n=2, N=2, i=0, j=0$ · Output: 6
- Input: $m=1, n=3, N=3, i=0, j=1$ · Output: 12

```
C
1 #include <stdio.h>
2
3 int dp[55][55][55];
4
5 int solve(int m, int n, int N, int i, int j)
6 {
7     if (i < 0 || i >= m || j < 0 || j >= n)
8         return 1;
9
10    if (N == 0)
11        return 0;
12
13    if (dp[i][j][N] != -1)
14        return dp[i][j][N];
15
16    dp[i][j][N] =
17        solve(m, n, N - 1, i + 1, j) +
18        solve(m, n, N - 1, i - 1, j) +
19        solve(m, n, N - 1, i, j + 1) +
20        solve(m, n, N - 1, i, j - 1);
21
22    return dp[i][j][N];
23 }
24
25 int main()
26 {
27     int m, n, N, i, j;
28
29     printf("Enter m n N startRow startCol: ");
30     scanf("%d%d%d%d", &m, &n, &N, &i, &j);
31
32     for (int x = 0; x < 55; x++)
33         for (int y = 0; y < 55; y++)
34             for (int z = 0; z < 55; z++)
35                 dp[x][y][z] = -1;
36
37     printf("Ways = %d", solve(m, n, N, i, j));
38
39     return 0;
40 }
```

2 2 2 0 0

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 2.444 Mb

Sample Input

2 2 2 0 0

Your Output

Enter m n N startRow startCol: Ways = 6

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

```
#include <stdio.h>
int rob(int arr[], int start, int end)
{
    int prev1 = 0, prev2 = 0;
    for (int i = start; i <= end; i++)
    {
        int temp = prev1;
        if (prev2 + arr[i] > prev1)
            prev1 = prev2 + arr[i];
        prev2 = temp;
    }
    return prev1;
}
int main()
{
    int n;
    printf("Enter number of houses: ");
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    if (n == 1)
    {
        printf("%d", arr[0]);
        return 0;
    }
    int ans1 = rob(arr, 0, n - 2);
    int ans2 = rob(arr, 1, n - 1);
    printf("Maximum Money = %d", ans1 > ans2 ? ans1 : ans2);
}
```

3

2 3 2

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.804 Mb

Sample Input

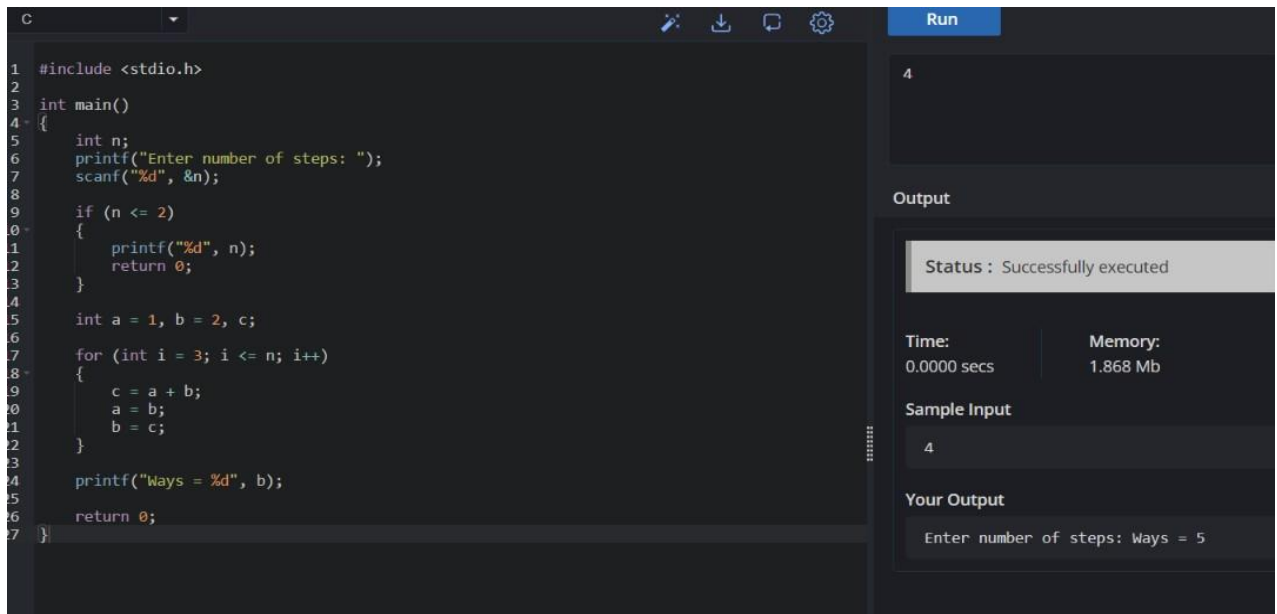
3

2 3 2

Your Output

Enter number of houses: Maximum Money = 3

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?



The screenshot shows a C program in a code editor. The program prompts the user to enter the number of steps, reads the input, and calculates the number of distinct ways to climb the stairs using a sliding window of three variables (a, b, c). The output panel shows the input '4' and the output 'Ways = 5'. The status is 'Successfully executed' with a time of 0.0000 secs and memory of 1.868 Mb.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n;
6     printf("Enter number of steps: ");
7     scanf("%d", &n);
8
9     if (n <= 2)
10    {
11        printf("%d", n);
12        return 0;
13    }
14
15    int a = 1, b = 2, c;
16    for (int i = 3; i <= n; i++)
17    {
18        c = a + b;
19        a = b;
20        b = c;
21    }
22
23    printf("Ways = %d", b);
24
25    return 0;
26 }
27
```

Output: 4

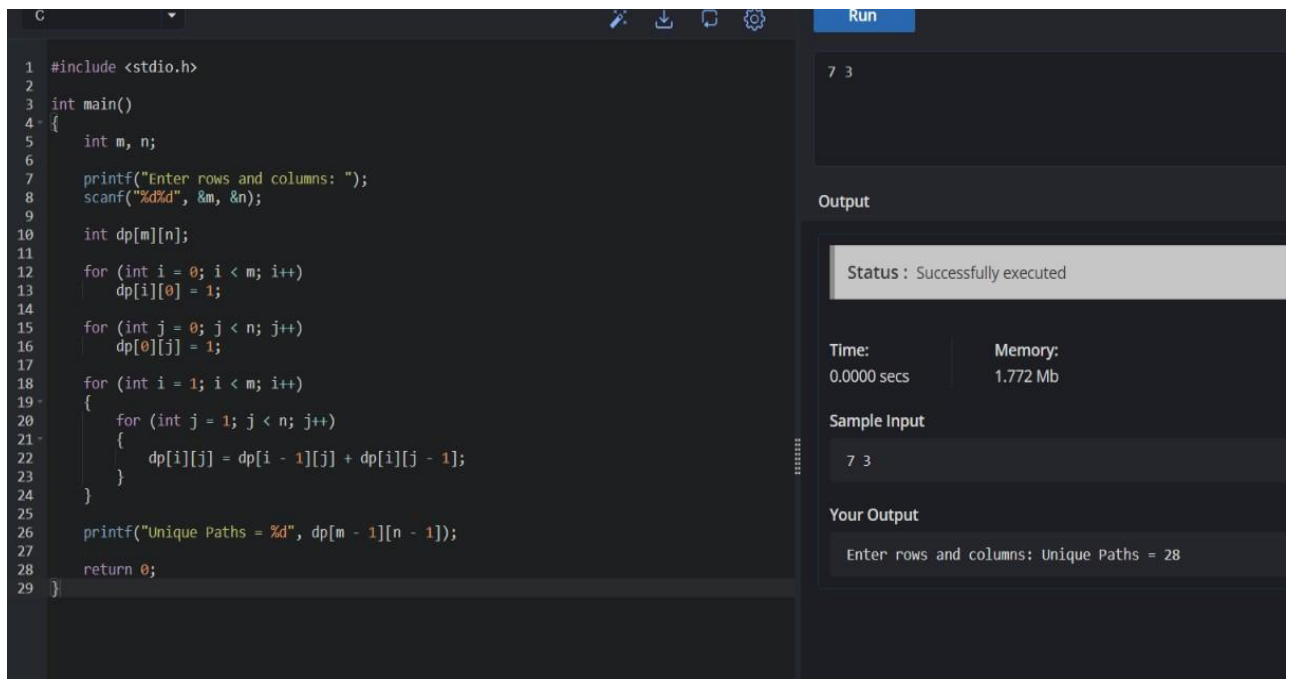
Status: Successfully executed

Time: 0.0000 secs Memory: 1.868 Mb

Sample Input: 4

Your Output: Enter number of steps: Ways = 5

14. A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?



The screenshot shows a C program using dynamic programming to solve the unique paths problem. It initializes a DP table and iterates through the grid to calculate the number of unique paths from the top-left to the bottom-right corner. The output panel shows the input '7 3' and the output 'Unique Paths = 28'. The status is 'Successfully executed' with a time of 0.0000 secs and memory of 1.772 Mb.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int m, n;
6
7     printf("Enter rows and columns: ");
8     scanf("%d%d", &m, &n);
9
10    int dp[m][n];
11
12    for (int i = 0; i < m; i++)
13        dp[i][0] = 1;
14
15    for (int j = 0; j < n; j++)
16        dp[0][j] = 1;
17
18    for (int i = 1; i < m; i++)
19    {
20        for (int j = 1; j < n; j++)
21        {
22            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
23        }
24    }
25
26    printf("Unique Paths = %d", dp[m - 1][n - 1]);
27
28    return 0;
29 }
```

Output: 7 3

Status: Successfully executed

Time: 0.0000 secs Memory: 1.772 Mb

Sample Input: 7 3

Your Output: Enter rows and columns: Unique Paths = 28

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like $s = \text{"abbxxxxzzy"}$ has the groups "a" , "bb" , "xxxx" , "zz" , and

"yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char s[100];
7
8      printf("Enter string: ");
9      scanf("%s", s);
10
11     int len = strlen(s);
12
13     for (int i = 0; i < len; i++)
14     {
15         int start = i;
16
17         while (i < len && s[i] == s[start])
18             i++;
19
20         if (i - start >= 3)
21             printf("[%d,%d]\n", start, i - 1);
22     }
23
24     return 0;
25 }

```

Output: abxxxxzy

Status: Successfully executed

Time: 0.0000 secs Memory: 1.74 Mb

Sample Input: abxxxxzy

Your Output: Enter string: [3,6]

16. The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules Any live cell with fewer than two live neighbors dies as if caused by under- population.

- Any live cell with two or three live neighbors lives on to the next generation.
- Any live cell with more than three live neighbors dies, as if by over- population.
- Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

```

19  int dx[] = {-1,-1,-1,0,0,1,1,1};
20  int dy[] = {-1,0,1,-1,1,-1,0,1};
21
22  for (int i = 0; i < m; i++)
23  {
24      for (int j = 0; j < n; j++)
25      {
26          int live = 0;
27
28          for (int k = 0; k < 8; k++)
29          {
30              int x = i + dx[k];
31              int y = j + dy[k];
32
33              if (x >= 0 && x < m && y >= 0 && y < n)
34                  live += board[x][y];
35          }
36
37          if (board[i][j] == 1)
38          {
39              if (live == 2 || live == 3)
40                  next[i][j] = 1;
41              else
42                  next[i][j] = 0;
43          }
44          else
45          {
46              if (live == 3)
47                  next[i][j] = 1;
48              else
49                  next[i][j] = 0;
50          }
51      }
52  }
53
54  printf("Next State:\n");
55
56  for (int i = 0; i < m; i++)
57  {
58      for (int j = 0; j < n; j++)
59          printf("%d ", next[i][j]);
60  }

```

Output: 4 3

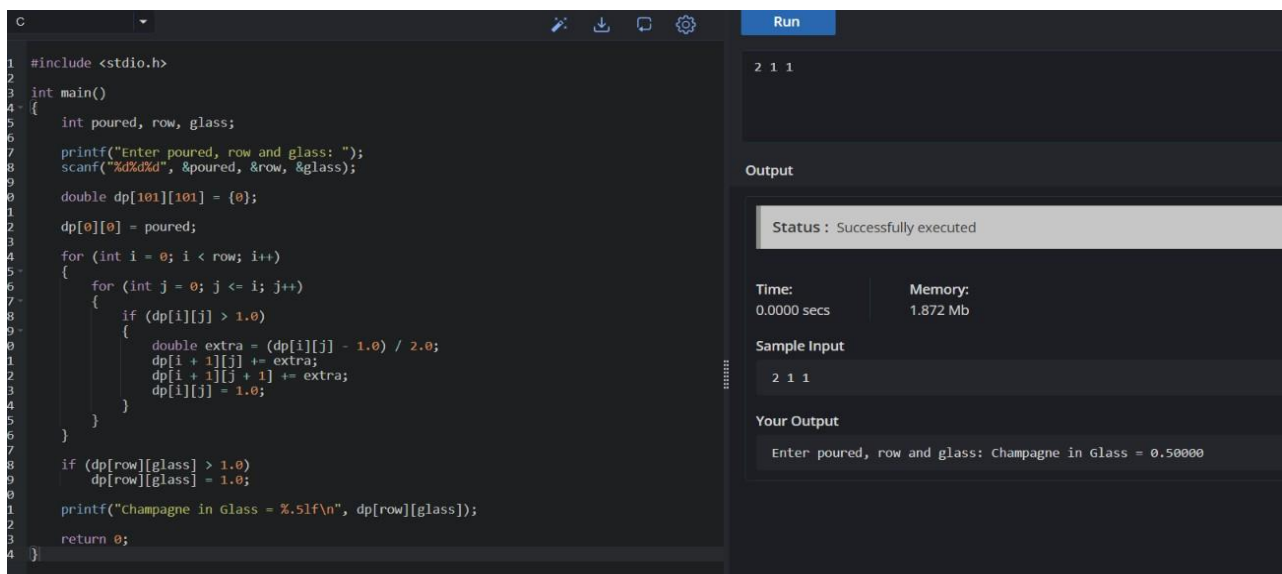
Status: Successfully executed

Time: 0.0000 secs Memory: 1.864 Mb

Sample Input: 4 3
0 1 0
0 0 1
1 1 1
0 0 0

Your Output: Enter rows and columns: Enter board:
Next State:
0 0 0
1 0 1
0 1 1
0 1 0

17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100 th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full.



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int poured, row, glass;
6
7     printf("Enter poured, row and glass: ");
8     scanf("%d%d%d", &poured, &row, &glass);
9
10    double dp[101][101] = {0};
11
12    dp[0][0] = poured;
13
14    for (int i = 0; i < row; i++)
15    {
16        for (int j = 0; j <= i; j++)
17        {
18            if (dp[i][j] > 1.0)
19            {
20                double extra = (dp[i][j] - 1.0) / 2.0;
21                dp[i + 1][j] += extra;
22                dp[i + 1][j + 1] += extra;
23                dp[i][j] = 1.0;
24            }
25        }
26    }
27
28    if (dp[row][glass] > 1.0)
29        dp[row][glass] = 1.0;
30
31    printf("Champagne in Glass = %.5lf\n", dp[row][glass]);
32    return 0;
33 }
```

Run

2 1 1

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.872 Mb

Sample Input

2 1 1

Your Output

Enter poured, row and glass: Champagne in Glass = 0.50000